



Developing BookStore V-1.5

- In 1.5 Version of BookStore, we will add the following skills learned.
 - Hystrix
 - Zuul
 - ELK Stack

Applying Hystrix in BookStore – V-1.5

A) Applying Hystix in BookPriceMS

1) Change the Spring Cloud Version in pom.xml

```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
```

2) Change the Spring Boot Version in pom.xml

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>
```

3) Add the following dependencies in pom.xml

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

4) Add the @EnableCircuitBreaker the main Spring Application class

```
@SpringBootApplication
@EnableSwagger2
@EnableEurekaClient
@EnableCircuitBreaker
public class BookPriceApplication {

}
```



- 5) Decide the method or Methods where you want to provide fallback mechanism and Specify the fallback method with @HystrixCommand

Now Update BookPriceController.java

```
@GetMapping("/bookPrice/{bookId}")
@HystrixCommand(fallbackMethod = "getBookPriceFallback")
public BookPrice getBookPrice(@PathVariable Integer bookId) {
....

}

@GetMapping("/offeredPrice/{bookId}")
@HystrixCommand(fallbackMethod = "getOfferedPriceFallback")
public double getOfferedPrice(@PathVariable Integer bookId) {
....

}
```

- 6) Write the Fallback methods with an Alternative Logic.

```
public BookPrice getBookPriceFallback(Integer bookId) {
....

}

public double getOfferedPriceFallback(Integer bookId) {
...

}
```

Note : I have Applied Hystrix Only in BookPriceMS.



Applying Zuul in BookStore – V-1.5

A) Setup Zuul API Gatewayserver in BookStore EcoSystem V-1.5

1) Copy and Import Zuul API Gateway Server into BookStore 1.5 Workspace

2) Add the Required Routes in application.properties

```
zuul.routes.book-price.path=/myjlc-book-price/**  
zuul.routes.book-price.url=MyBookPriceMS
```

```
zuul.routes.book-search.path=/myjlc-book-search/**  
zuul.routes.book-search.url=MyBookSearchMS
```

```
zuul.routes.book-store.path=/myjlc-book-store/**  
zuul.routes.book-store.url=MyBookStoreWeb
```

3) Remaining Almost Same. Check the Code given in Later Pages

B) Applying Zuul in BookPriceMS

4) Change the Spring Cloud Version in pom.xml

```
<properties>  
<java.version>1.8</java.version>  
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>  
</properties>
```

5) Change the Spring Boot Version in pom.xml

```
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.3.3.RELEASE</version>  
</parent>
```

6) Add the Zuul dependency in pom.xml

```
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>  
</dependency>
```



C) Applying Zuul in BookSearchMS

7) Change the Spring Cloud Version in pom.xml

```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
```

8) Change the Spring Boot Version in pom.xml

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>
```

9) Add the Zuul dependency in pom.xml

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>
```

10) Update the BookPriceProxy in BookSearchMS

```
@FeignClient("MyApiGatewayServer")
```

```
public interface BookPriceProxy {
```

```
    @GetMapping("/myjlc-book-price/bookPrice/{bookId}")
```

```
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);
```

```
    @GetMapping("/myjlc-book-price/offeredPrice/{bookId}")
```

```
    public double getOfferedPrice(@PathVariable Integer bookId);
```

```
}
```

D) Applying Zuul in BookStoreWeb

11) Change the Spring Cloud Version in pom.xml

```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
```

12) Change the Spring Boot Version in pom.xml

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>
```

13) Add the Zuul dependency in pom.xml

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>
```

14) Update the BookSearchProxy in BookStoreWeb

```
@FeignClient("MyApiGatewayServer")
```

```
public interface BookSearchProxy {
```

```
    @GetMapping("/myjlc-book-price/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);
```

```
    @GetMapping("/myjlc-book-price/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);
```

```
    @GetMapping("/myjlc-book-search/mybooks/{author}/{category}")
    public List<Book> getBooks(@PathVariable String author, @PathVariable
String category);
```

```
    @GetMapping("/myjlc-book-search/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId);
```

```
}
```



15) Remove BookPriceProxy in BookStoreWeb

- 16) No Updates on PlaceOrderProxy and UserRatingProxy in BookStoreWeb** because
We don't want to Intercept the Requests coming to PlaceOrder and UserRating

Applying ELK Stack in BookStore – V-1.5

A) Using Logging in BookPrice MS V-1.5

1) Update the application.properties of BookPriceMS

```
logging.file.name=E:/mylogs/jlc-book-price.log
```

**2) Add the Log Statements in Controller and Services of not there.
Avoid S.O.P's**

```
//Create the logger Instance
```

```
static Logger mylogger = LoggerFactory.getLogger(BookPriceController.class);
```

```
//Log the message
```

```
mylogger.info("---BookPriceController---getBookPrice()-----");
```

B) Using Logging in BookSearch MS V-1.5

3) Update the application.properties of BookSearchMS

```
logging.file.name=E:/mylogs/jlc-book-search.log
```

**4) Add the Log Statements in Controller and Services of not there.
Avoid S.O.P's**

```
//Create the logger Instance
```

```
static Logger mylogger = LoggerFactory.getLogger(BookController.class);
```

```
//Log the message
```

```
mylogger.info("---BookController---getBooks()-----");
```



C) Using Logging in BookStoreWeb V-1.5

5) Update the application.properties of BookStoreWeb

logging.file.name=E:/mylogs/jlc-book-store.log

6) Add the Log Statements in Controller and Services of not there.

Avoid S.O.P's

D) Using Logging in PlaceOrderMS V-1.5

7) Update the application.properties of PlaceOrderMS

logging.file.name=E:/mylogs/jlc-place-order.log

8) Add the Log Statements in Controller and Services of not there.

Avoid S.O.P's

E) Using Logging in UserRating MS V-1.5

9) Update the application.properties of UserRatingMS

logging.file.name=E:/mylogs/jlc-user-rating.log

10) Add the Log Statements in Controller and Services of not there.

Avoid S.O.P's

F) Using Logging in GateWayServer V-1.5

11) Update the application.properties of Gateway Server

logging.file.name=E:/mylogs/jlc-gate-way.log

12) Add the Log Statements in Controller and Services of not there.

Avoid S.O.P's



G) Prepare the logstash configuration file as follows

jlc-logstash.conf

```
input {  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-price.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-search.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-store.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-place-order.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
}
```



```
file {
  type => "java"
  path => "E:/mylogs/jlc-user-rating.log"
  codec => multiline {
    pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"
    negate => "true"
    what => "previous"
  }
}
file {
  type => "java"
  path => "E:/mylogs/jlc-gate-way.log"
  codec => multiline {
    pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"
    negate => "true"
    what => "previous"
  }
}

filter {
  if [message] =~ "\tat" {
    grok {
      match => ["message", "^(\\tat)"]
      add_tag => ["my-jlc-error-trace"]
    }
  }
}

output {
  stdout {
    codec => rubydebug
  }
}

elasticsearch {
  hosts => ["localhost:9200"]
}
}
```

Steps to Run BookStore-1.5

1) Start MySQL Server If not Started.

2) Start the RabbitMQ Server If not Started.

E:\MyRabbitMQ\rabbitmq_server-3.9.5\sbin>rabbitmq-server.bat

3) Start the ElasticSearch

E:\elasticsearch-7.15.2/bin>elasticsearch.bat

4) Start the Kibana

E:\kibana-7.15.2/bin>kibana.bat

5) Start the logStash

E:\logstash-7.15.2/bin> logstash -f jlc-logstash.conf

6) Start BookAdmin Server

7) Start Eureka Server

8) Start Gateway Server

9) Start BookPriceMS

10) Start BookSearchMS

11) Start PlaceOrderMS

12) Start UserRatingMS

13) Start BookStoreWeb

http://localhost:5000/

http://localhost:5678/myjlc-book-store/



BookPriceMS – V-1.5: Files required

1. BookPriceController.java	Updated in V1.5
2. BookPriceException.java	Newly Added in V.1.5
3. BookPriceService.java	Same as V.1.4
4. BookPriceServiceImpl.java	Same as V.1.4
5. BookPriceDAO.java	Same as V.1.4
6. BookPrice.java	Same as V.1.4
7. JLCBookPriceConfig.java	Same as V.1.4
8. BookPriceApplication.java	Updated in V1.5
9. application.properties	Updated in V1.5
10.pom.xml	Updated in V1.5

1. BookPriceController.java

```
package com.jlcindia.bookprice;

import java.io.PrintWriter;
import java.io.StringWriter;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import io.swagger.annotations.ApiOperation;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookPriceController {
    static Logger mylogger = LoggerFactory.getLogger(BookPriceController.class);
```



@Autowired

BookPriceService bookPriceService;

@GetMapping("/bookPrice/{bookId}")

@ApiOperation(value = " getBookPrice", response = BookPrice.class, notes = "Gives BookPrice Object of given bookId")

@HystrixCommand(fallbackMethod = "getBookPriceFallback")

public BookPrice getBookPrice(@PathVariable Integer bookId) {
mylogger.info("---BookPriceController---getBookPrice()-----");

if(1==1) {

try {

throw new BookPriceException();

}catch(BookPriceException ex) {

ex.printStackTrace();

mylogger.error(ex.toString());

StringWriter sw = new StringWriter();

PrintWriter pw = new PrintWriter(sw);

ex.printStackTrace(pw);

String stackTrace = sw.toString();

mylogger.error("MyException - " + stackTrace);

}

}

return null;

}

@GetMapping("/offeredPrice/{bookId}")

@ApiOperation(value = " getOfferedPrice", response = double.class, notes = "Gives Offered Price of given bookId")

@HystrixCommand(fallbackMethod = "getOfferedPriceFallback")

public double getOfferedPrice(@PathVariable Integer bookId) {

mylogger.info("---BookPriceController---getOfferedPrice()-----");

if(1==1) {

try {

throw new BookPriceException();

}catch(BookPriceException ex) {

ex.printStackTrace();

mylogger.error(ex.toString());



```
StringWriter sw = new StringWriter();
PrintWriter pw = new PrintWriter(sw);
ex.printStackTrace(pw);
String stackTrace = sw.toString();
mylogger.error("MyException - " + stackTrace);
}
}

return 0.0;
}

public BookPrice getBookPriceFallback(Integer bookId) {
mylogger.info("---BookPriceController---getBookPriceFallback()-----");
BookPrice bookPrice =
bookPriceService.getBookPriceById(bookId);
return bookPrice;
}

public double getOfferedPriceFallback(Integer bookId) {
mylogger.info("---BookPriceController---getOfferedPriceFallback()-----");
double offeredPrice =
bookPriceService.getOfferedPriceById(bookId);
return offeredPrice;
}
}
```

2. BookPriceException.java

```
package com.jlcindia.bookprice;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookPriceException extends Exception{

}
```



8. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableSwagger2
@EnableEurekaClient
@EnableCircuitBreaker
public class BookPriceApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookPriceApplication.class, args);
    }
}
```

9. application.properties

```
spring.application.name=MyBookPriceMS

server.port=9000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbookpricedb
spring.datasource.username=root
spring.datasource.password=srinivas
```



```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

logging.file.name=E:/mylogs/jlc-book-price.log

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

```
# INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

10. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookprice</groupId>
<artifactId>BookPriceMS</artifactId>
<version>1.5</version>
<packaging>jar</packaging>

<name>MyBookPriceMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookPrice MS developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>
```



```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>de.codecentric</groupId>
```

```
<artifactId>spring-boot-admin-starter-client</artifactId>
```

```
<version>2.3.0</version>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-actuator</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>mysql</groupId>
```

```
<artifactId>mysql-connector-java</artifactId>
```

```
<version>5.1.6</version>
```

```
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<finalName>MyBookPriceMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

BookSearchMS – V-1.5: Files required

1. BookController.java	Same as V1.4
2. BookPriceInfo.java	Same as V1.4
3. BookInfo.java	Same as V1.4
4. BookInventoryInfo.java	Same as V1.4
5. BookRatingInfo.java	Same as V1.4
6. BookPriceProxy.java	Updated in V1.5
7. BookService.java	Same as V1.4
8. BookServiceImpl.java	Same as V1.4
9. BookDAO.java	Same as V1.4
10. BookRatingDAO.java	Same as V1.4
11. BookInventoryDAO.java	Same as V1.4
12. Book.java	Same as V1.4
13. BookRating.java	Same as V1.4
14. BookInventory.java	Same as V1.4
15. JLCBookSearchConfig.java	Same as V1.4
16. BookSearchApplication.java	Same as V1.4
17.application.properties	Updated in V1.5
18.pom.xml	Updated in V1.5

6.BookPriceProxy.java

```
package com.jlcindia.booksearch;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient("MyApiGatewayServer")
public interface BookPriceProxy {

    @GetMapping("/myjlc-book-price/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

    @GetMapping("/myjlc-book-price/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);
}
```



17. application.properties

spring.application.name=**MyBookSearchMS**

server.port=**8000**

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

eureka.client.healthcheck.enabled=true

spring.rabbitmq.host=localhost

spring.rabbitmq.port=5672

spring.rabbitmq.username=guest

spring.rabbitmq.password=guest

spring.boot.admin.client.url=http://localhost:5555

management.endpoints.web.exposure.include=*

management.endpoint.health.show-details=always

spring.datasource.driver-class-name=com.mysql.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb

spring.datasource.username=root

spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000

spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-search.log

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=update

INFO ENDPOINT CONFIGURATION

info.app.name=@project.name@

info.app.description=@project.description@

info.app.version=@project.version@

info.app.encoding=@project.build.sourceEncoding@

info.app.java.version=@java.version@

info.app.developer=Srinivas Dande

info.app.company=Java Learning Center



18. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.5</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookSeach MS developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<finalName>MyBookSearchMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



PlaceOrderMS – V-1.5: Files required

1. OrderController.java	Same as V1.4
2. OrderFullInfo.java	Same as V1.4
3. OrderInfo.java	Same as V1.4
4. OrderItemInfo.java	Same as V1.4
5. BookInventoryInfo.java	Same as V1.4
6. OrderService.java	Same as V1.4
7. OrderServiceImpl.java	Same as V1.4
8. OrderDAO.java	Same as V1.4
9. OrderItemDAO.java	Same as V1.4
10. BookInventoryDAO.java	Same as V1.4
11. Order.java	Same as V1.4
12. OrderItem.java	Same as V1.4
13. BookInventory.java	Same as V1.4
14. JLCPlaceOrderConfig.java	Same as V1.4
15. PlaceOrderApplication.java	Same as V1.4
16.application.properties	Updated in V1.5
17. pom.xml	Same as V1.4

16. application.properties

spring.application.name=MyPlaceOrderMS

server.port=7000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost

spring.rabbitmq.port=5672

spring.rabbitmq.username=guest

spring.rabbitmq.password=guest

management.endpoints.web.exposure.include=*

management.endpoint.health.show-details=always



```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
spring.datasource.username=root
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-place-order.log
```

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

```
# INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

UserRatingMS – V-1.5: Files required

1. RatingController.java	Same as V1.4
2. UserRatingInfo.java	Same as V1.4
3. BookRatingInfo.java	Same as V1.4
4. RatingService.java	Same as V1.4
5. RatingServiceImpl.java	Same as V1.4
6. UserRatingDAO.java	Same as V1.4
7. BookRatingDAO.java	Same as V1.4
8. UserRating.java	Same as V1.4
9. BookRating.java	Same as V1.4
10. JLCUserRatingConfig.java	Same as V1.4
11. UserRatingApplication.java	Same as V1.4
12.application.properties	Updated in V1.5
13. pom.xml	Same as V1.4



12. application.properties

spring.application.name=**MyUserRatingMS**

server.port=**6500**

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-user-rating.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center



BookStoreWeb – V-1.5: Files required

1. index.jsp	Same as V1.4
2. myheader.jsp	Same as V1.4
3. myfooter.jsp	Same as V1.4
4. showBooksList.jsp	Same as V1.4
5. showBookInfo.jsp	Same as V1.4
6. showCart.jsp	Same as V1.4
7. orderSuccess.jsp	Same as V1.4
8. addRating.jsp	Same as V1.4
9. ratingSuccess.jsp	Same as V1.4
10. ratingsList.jsp	Same as V1.4
11. bookstore.css	Same as V1.4
12. Book.java	Same as V1.4
13. BookInfo.java	Same as V1.4
14. Order.java	Same as V1.4
15. OrderItem.java	Same as V1.4
16. UserRating.java	Same as V1.4
17. BookStoreController.java	Same as V1.4
18. OrderFullInfo.java	Same as V1.4
19. OrderInfo.java	Same as V1.4
20. OrderItemInfo.java	Same as V1.4
21. UserRatingInfo.java	Same as V1.4
22. BookPriceInfo.java	Same as V1.4
23. BookSearchProxy.java	Updated in V1.5
24. PlaceOrderProxy.java	Same as V1.4
25. UserRatingProxy.java	Same as V1.4
26. BookStoreService.java	Same as V1.4
27. BookStoreServiceImpl.java	Updated in V1.5
28. JLCBookStoreWebConfig.java	Same as V1.4
29. BookStoreWebApplication.java	Updated in V1.4
30. application.properties	Updated in V1.5
31. pom.xml	Updated in V1.5



23. BookSearchProxy.java

```
package com.jlcindia.bookstoreweb;

import java.util.List;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient("MyApiGatewayServer")
public interface BookSearchProxy {

    @GetMapping("/myjlc-book-price/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

    @GetMapping("/myjlc-book-price/offeredPrice/{bookId}")
    public double getOfferedPrice(@PathVariable Integer bookId);

    @GetMapping("/myjlc-book-search/mybooks/{author}/{category}")
    public List<Book> getBooks(@PathVariable String author, @PathVariable String category);

    @GetMapping("/myjlc-book-search/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId);
}
```

27. BookStoreServiceImpl.java

```
package com.jlcindia.bookstoreweb;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
```



```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.jlcindia.rabbitmq.OrderFullInfo;
import com.jlcindia.rabbitmq.OrderInfo;
import com.jlcindia.rabbitmq.OrderItemInfo;
import com.jlcindia.rabbitmq.UserRatingInfo;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Service
public class BookStoreServiceImpl implements BookStoreService {

    static Logger log = LoggerFactory.getLogger(BookStoreServiceImpl.class);

    Map<Integer, Book> booksMap = new LinkedHashMap<>();

    @Autowired
    RabbitTemplate rabbitTemplate;

    @Autowired
    BookSearchProxy bookSearchProxy;

    @Autowired
    PlaceOrderProxy placeOrderProxy;

    @Autowired
    UserRatingProxy userRatingProxy;

    public List<String> getAuthorsList() {
        List<String> authorsList = new ArrayList<>();
        authorsList.add("All Authors");
        authorsList.add("Srinivas");
        authorsList.add("Vas");
        authorsList.add("Sri");
        return authorsList;
    }
}
```



```
public List<String> getCategoryList() {
    List<String> catList = new ArrayList<>();
    catList.add("All Categories");
    catList.add("Web");
    catList.add("Spring");
    return catList;
}

public List<Book> getMyBooks(String author, String category) {
    System.out.println("BookStoreServiceImpl - getBooks()");

    if (author == null || author.length() == 0) {
        author = "All Authors";
    }
    if (category == null || category.length() == 0) {
        category = "All Categories";
    }

    //Invoking BookSearch Rest API with Feign
    List<Book> bookList = bookSearchProxy.getBooks(author, category);

    return bookList;
}

public BookInfo getBookInfoByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookInfoByBookId()");

    //Invoking BookSearch Rest API with Feign
    BookInfo bookInfo = bookSearchProxy.getBookById(bookId);

    return bookInfo;
}

public Book getBookByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookByBookId()");
    System.out.println(bookId);
    Book mybook = booksMap.get(bookId);

    return mybook;
}
```



```
public void placeOrder(Map<Integer, Book> mycartMap) {

    // New Code
    System.out.println("---2.BookStoreServiceImpl--placeOrder()----");
    List<OrderItemInfo> itemList = new ArrayList<>();
    double totalPrice = 0.0;
    int totalQuantity = 0;

    for (Book mybook : mycartMap.values()) {
        Integer bookId = mybook.getBookId();

        // Invoke BookPriceMS Rest API with Feign
        double offerPrice = bookSearchProxy.getOfferedPrice(bookId);

        OrderItemInfo item = new OrderItemInfo(0, bookId, 1, offerPrice);
        itemList.add(item);

        totalPrice = totalPrice + offerPrice;
        totalQuantity = totalQuantity + 1;
    }

    Date today = Calendar.getInstance().getTime();
    SimpleDateFormat formatter = new SimpleDateFormat("dd-MM-yyyy hh:mm");
    String orderDate = formatter.format(today);
    System.out.println(orderDate);

    OrderInfo orderInfo = new OrderInfo(orderDate, "U-111", totalQuantity, totalPrice, "New");

    OrderFullInfo orderFullInfo = new OrderFullInfo();
    orderFullInfo.setOrder(orderInfo);
    orderFullInfo.setItemsList(itemList);

    // Sending Order Message to RabbitMQ
    rabbitTemplate.convertAndSend("myorder.exchange","myorder.key", orderFullInfo);
    System.out.println("Order Placed");
    }

    @Override
    public List<Order> getMyOrders(String userId) {

        // Invoke PlaceOrderMS Rest API with Feign
        List<Order> myorders = placeOrderProxy.getOrdersByUserId(userId);
        return myorders;
    }
}
```



```
public void addUserRating(UserRating userRating) {
    System.out.println("---2.BookStoreServiceImpl--addUserRating()----");

    //Sending UserRating Message RabbitMQ
    UserRatingInfo userRatingInfo=new UserRatingInfo(userRating.getUserId(),
    userRating.getBookId(), userRating.getRating(), userRating.getReview());

    rabbitTemplate.convertAndSend("myuser.ratings.exchange","myuser.ratings.key",
    userRatingInfo);

    System.out.println("Rating Added");
}

public List<UserRating> getMyRatings(String userId) {

    // Invoke UserRatingMS Rest API with Feign
    List<UserRating> ratingsList = userRatingProxy.getUserRatingByUserId(userId);

    return ratingsList;
}
}
```

30. application.properties

```
spring.application.name=MyBookStoreWeb

server.port=5000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```



```
logging.file.name=E:/mylogs/jlc-book-store.log
```

```
# INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

31.pom.xml

```
<project ....">
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstoreweb</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.5</version>
<packaging>jar</packaging>

<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>

<dependency>
<groupId>>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>
```



```
<dependency>
<groupId>org.webjars</groupId>
<artifactId>bootstrap</artifactId>
<version>4.3.1</version>
</dependency>

<dependency>
<groupId>org.webjars</groupId>
<artifactId>jquery</artifactId>
<version>3.4.1</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<finalName>MyBookStoreWeb</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Steps to Run BookStore-1.5

1) Start MySQL Server If not Started.

2) Start the RabbitMQ Server If not Started.

E:\MyRabbitMQ\rabbitmq_server-3.9.5\sbin>rabbitmq-server.bat

3) Start the ElasticSearch

E:\elasticsearch-7.15.2/bin>elasticsearch.bat

4) Start the Kibana

E:\kibana-7.15.2/bin>kibana.bat

5) Start the logStash

E:\logstash-7.15.2/bin> logstash -f jlc-logstash.conf

6) Start BookAdmin Server

7) Start Eureka Server

8) Start Gateway Server

9) Start BookPriceMS

10) Start BookSearchMS

11) Start PlaceOrderMS

12) Start UserRatingMS

13) Start BookStoreWeb

http://localhost:5000/

http://localhost:5678/myjlc-book-store/